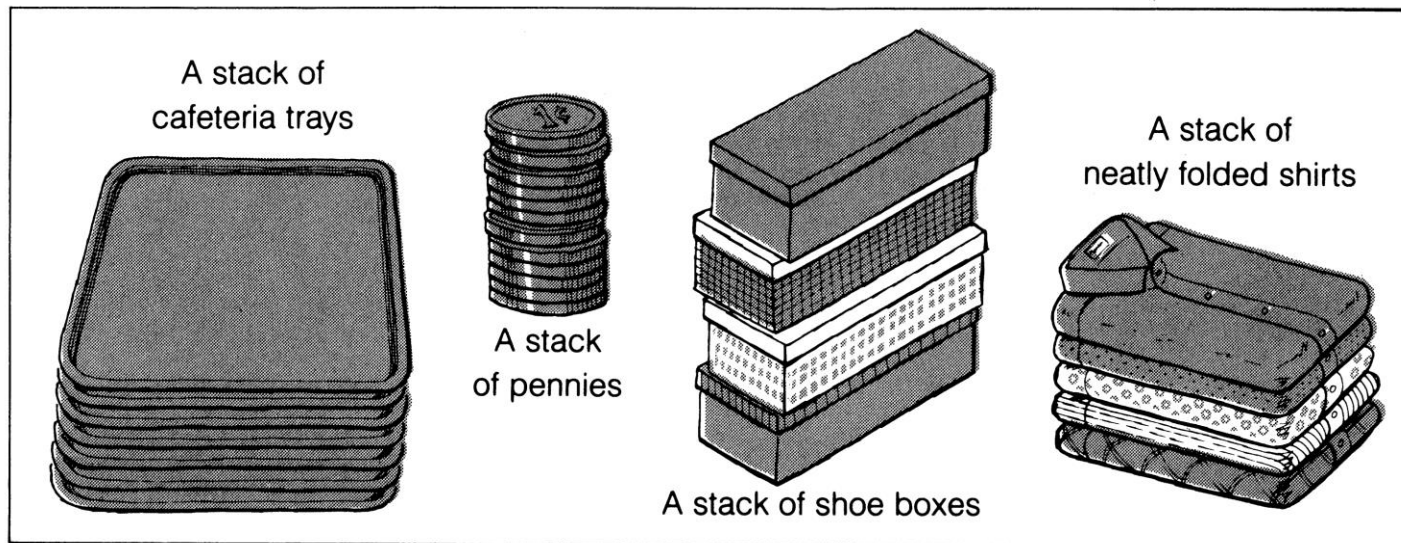


Data Structures & Algorithms (CS-212)

Stacks

Stack

- It is an ordered group of homogeneous items of elements.
- Real life



Top of stack
(accessible)

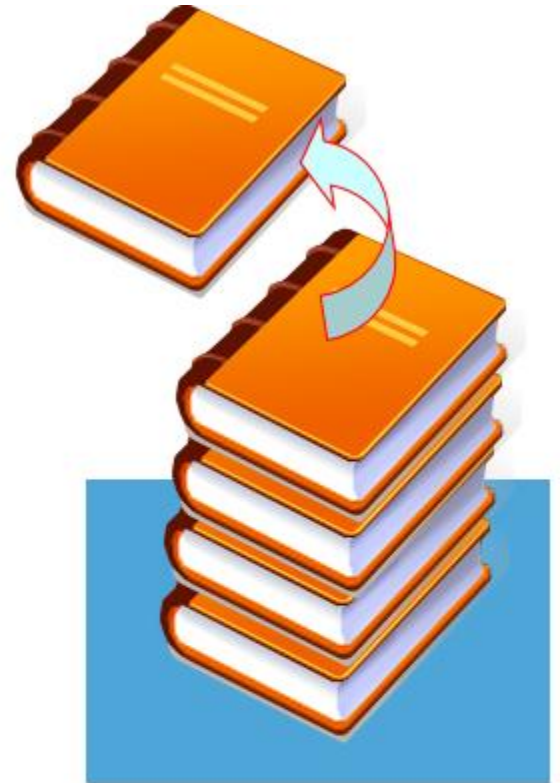


Bottom of stack
(inaccessible)

A **stack** of
four books



Push a new
book on top



Pop a book
from top

What is Stack

- Stack is a data structure of ordered items such that items can be inserted and removed only at one end.
- A *stack* is a container of objects that are inserted and removed according to the *Last-in First-out (LIFO)* principle.
- Objects can be inserted into a stack at any time, but only the most recently inserted (that is, “last”) object can be removed at any time.

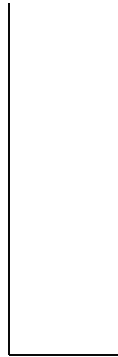
Applications

Example 5.1: *Internet Web browsers store the addresses of recently visited sites on a stack. Each time a user visits a new site, that site's address is “pushed” onto the stack of addresses. The browser then allows the user to “pop” back to previously visited sites using the “back” button.*

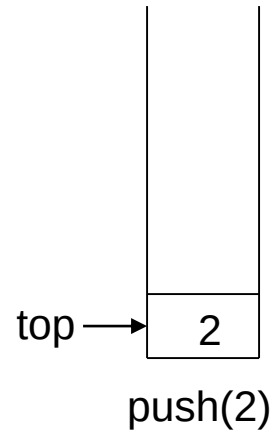
Applications

Example 5.2: *Text editors usually provide an “undo” mechanism that cancels recent editing operations and reverts to former states of a document. This undo operation can be accomplished by keeping text changes in a stack.*

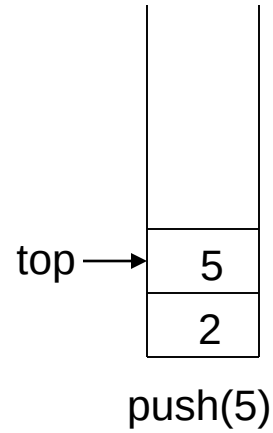
Stack Operations



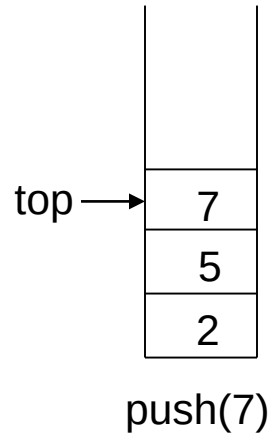
Stack Operations



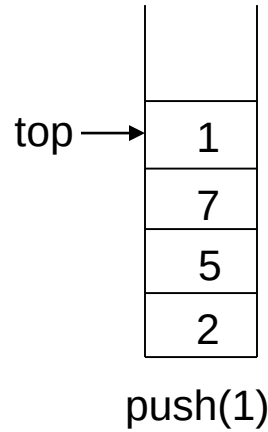
Stack Operations



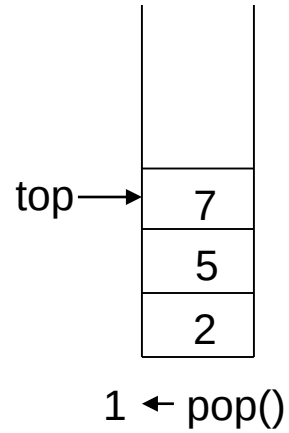
Stack Operations



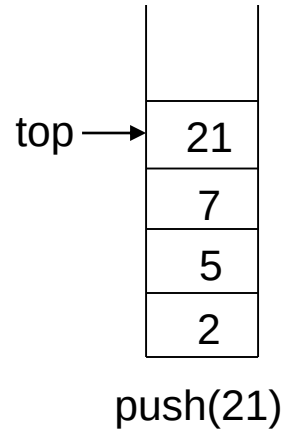
Stack Operations



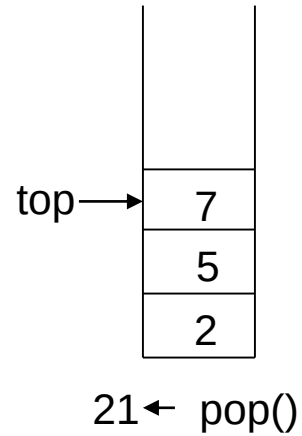
Stack Operations



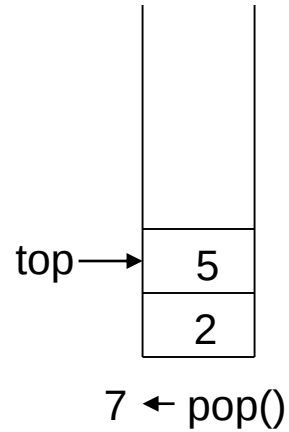
Stack Operations



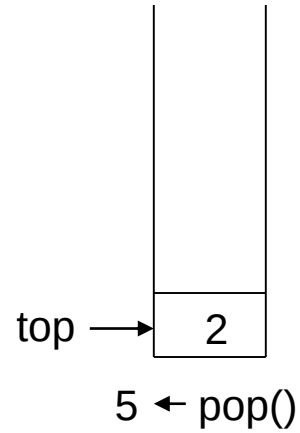
Stack Operations



Stack Operations

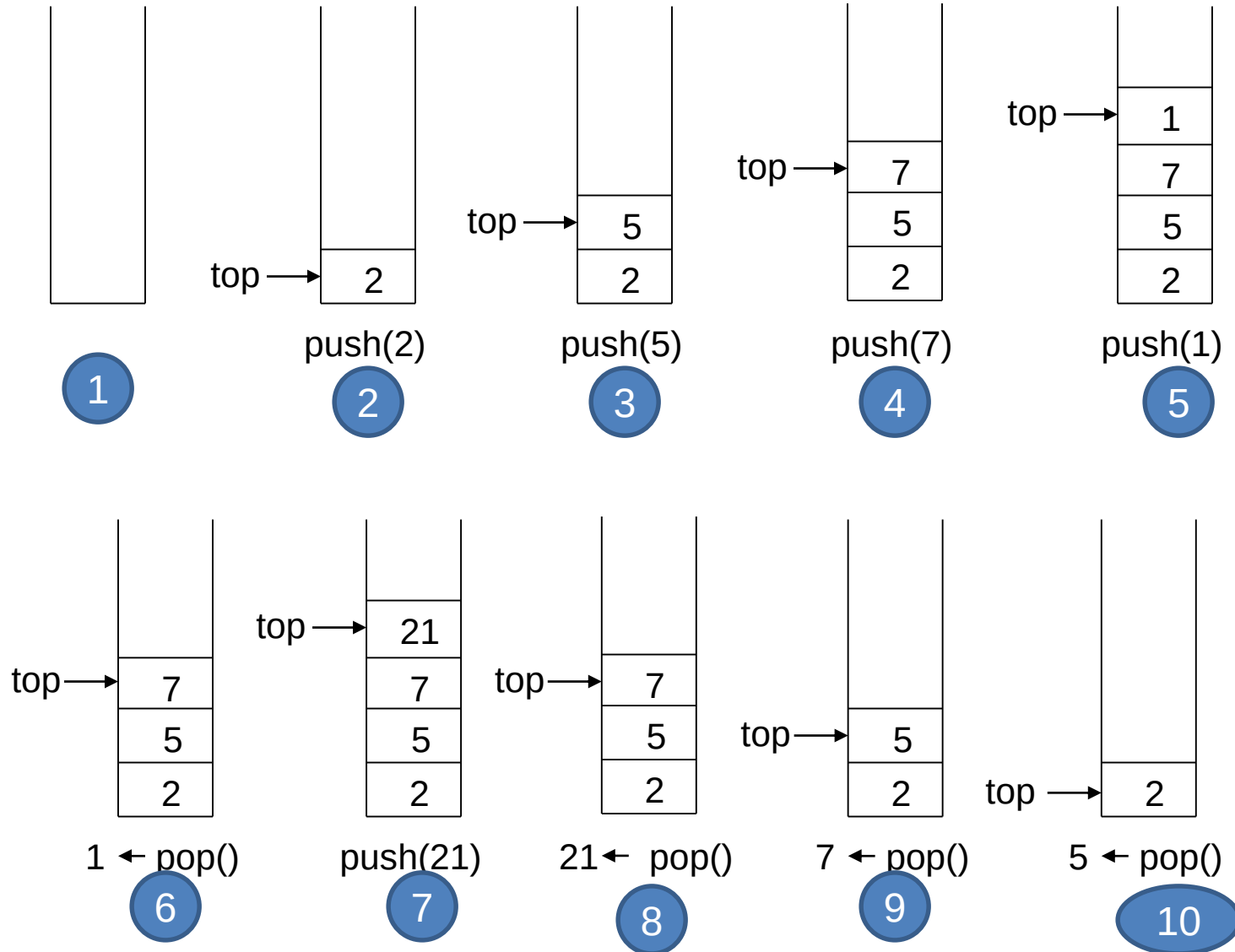


Stack Operations



Stack Operations

- 2- push (2)
- 3- push (5)
- 4- push (7)
- 5- push(1)
- 6- pop()
- 7- push(21)
- 8- pop()
- 9- pop()
- 10-pop()



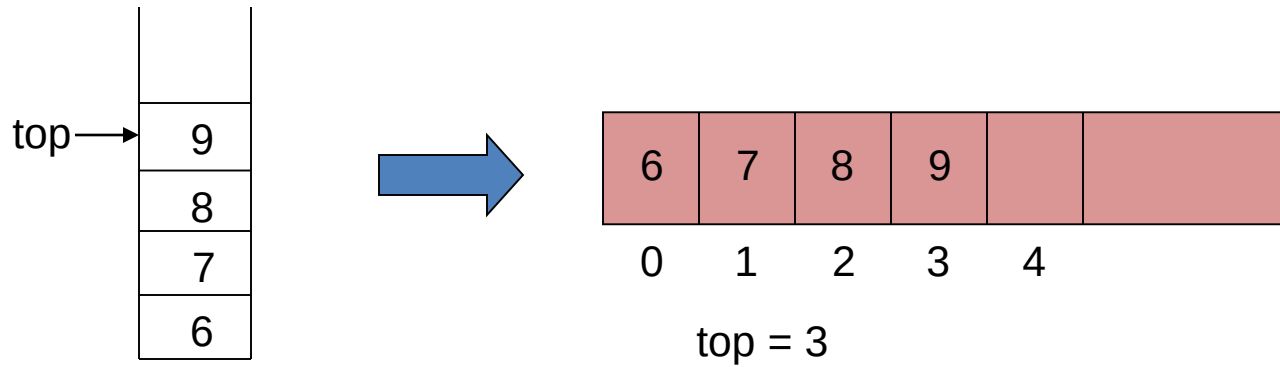
Implementing a Stack

- Two different ways to implement a stack
 - Array
 - Linked list
- Which method to use depends on the application
 - what advantages and disadvantages does each implementation have?

Implementing Stacks: Array

- Advantages
 - best performance
- Disadvantage
 - fixed size
- Basic implementation
 - initially empty array
 - field to record where the next data gets placed into
 - if array is full, push() returns false
 - otherwise adds it into the correct spot
 - if array is empty, pop() returns null
 - otherwise removes the next item in the stack

Stack using an Array



Stack using an Array

- In case of an array, it is possible that the array may “fill-up” if we push enough elements.
- Have a boolean function **IsFull()** which returns true if stack (array) is full, false otherwise.
- We would call this function before calling **push(x)**.

Stack Class

```
#define MAXSIZE 500 //max size for stack
```

```
class CStack
```

```
{
```

```
    private:
```

```
        int top;
```

```
    public:
```

```
        int integerStack[MAXSIZE]; //stack array
```

```
};
```

Stack Operations

```
#define MAXSIZE 500 //max size for stack
```

```
class CStack
```

```
{
```

```
    private:
```

```
        int top;
```

```
    public:
```

```
        int integerStack[MAXSIZE]; //stack array
```

```
        CStack() { top = -1; }           // Constructor
```

```
};
```

Stack Operations

- Push
- Pop
- IsEmpty
- IsFull
- Top

Stack Operations

```
#define MAXSIZE 500 //max size for stack
```

```
class CStack
{
    private:
        int top;
    public:
        int integerStack[MAXSIZE]; //stack array
        CStack() { top = -1; }      // Constructor
        bool push(int x);
        int pop();
        bool isEmpty();
        bool isFull();
        int top();
};
```

Push Operation

```
bool CStack::push(int item)
{
    if (top >= (MAXSIZE-1))
    {
        cout << "Stack Overflow!!!";
        return false;
    }
    else {
        integerStack[++top] = item;
        cout<<item<<endl;
        return true;
    }
}
```

Pop Operation

```
//removes or pops elements out of the stack
int CStack::pop()
{
    if (top < 0) {
        cout << "Stack Underflow!!";
        return 0;
    }
    else {
        int item = integerStack[top--];
        return item;
    }
}
```

IsEmpty Operation

//check if stack is empty

```
bool CStack::isEmpty()
```

```
{
```

```
    return (top < 0);
```

```
}
```

IsFull Operation

//check if stack is Full

```
bool CStack::isFull()
```

```
{  
    return ( top == MAXSIZE-1 ) ;  
  
}
```

Top Operation

```
int CStack::top()  
{  
    return integerStack[top];  
}
```

Driver Program

```
int main()
{
    CStack stack;
    cout<<"The Stack Push "<<endl;
    stack.push(12);
    stack.push(41);
    stack.push(6);
    cout<<"The Stack Pop : "<<endl;
    while(!stack.isEmpty())
    {
        cout<<stack.pop()<<endl;
    }
    return 0;
}
```

Lecture content adapted from Michael T. Goodrich textbook,
chapters 5.